

## ✓ Preparation Notebook

### ✓ Setup Environment

```
# DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd
```

```
from utstd.folders import *
from utstd.ipyrenders import *
```

```
at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()
```

```
import warnings
warnings.simplefilter(action='ignore')
```

```
_____ 12.9/12.9 MB 7.4 MB/s eta 0:00:00
_____ 4.9/4.9 MB 14.7 MB/s eta 0:00:00
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This b  
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.  
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.  
Mounted at /content/gdrive

You can now save your data files in: /content/gdrive/MyDrive/36106/assignment/AT3/data

### ✓ Student Information

```
group_name = "36106-26AU-AT3-Group 20"  
student_name = "Devvrat Charusmiti Joshi"  
student_id = "25657887"
```

```
# DO NOT MODIFY THE CODE IN THIS CELL  
print_tile(size="h1", key='group_name', value=group_name)
```

group\_name

**36106-26AU-AT3-Group 20**

```
# DO NOT MODIFY THE CODE IN THIS CELL  
print_tile(size="h1", key='student_name', value=student_name)
```

student\_name

**Devvrat Charusmiti Joshi**

```
# DO NOT MODIFY THE CODE IN THIS CELL  
print_tile(size="h1", key='student_id', value=student_id)
```

student\_id

**25657887**

### ✓ 0. Python Packages

#### 0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: `! pip install`  
`<package_name>`

## ✓ 0.b Import Packages

```
# DO NOT MODIFY THE CODE IN THIS CELL
import pandas as pd
import altair as alt
import numpy as np

pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)

alt.data_transformers.disable_max_rows()

DataTransformerRegistry.enable('default')
```

## A. Feature Selection

### ✓ A.0 Load Data

```
# DO NOT MODIFY THE CODE IN THIS CELL
# Load datasets
try:
    customer_df = pd.read_csv(at.folder_path / "customer.csv")
    person_df = pd.read_csv(at.folder_path / "person.csv")
    product_category_df = pd.read_csv(at.folder_path / "product_category.csv")
    product_cost_history_df = pd.read_csv(at.folder_path / "product_cost_history.csv")
    product_list_price_history_df = pd.read_csv(at.folder_path / "product_list_price_history.csv")
    product_sub_category_df = pd.read_csv(at.folder_path / "product_sub_category.csv")
    product_df = pd.read_csv(at.folder_path / "product.csv")
    sales_order_detail_df = pd.read_csv(at.folder_path / "sales_order_detail.csv")
    sales_order_header_df = pd.read_csv(at.folder_path / "sales_order_header.csv")
    sales_territory_df = pd.read_csv(at.folder_path / "sales_territory.csv")
    special_offer_product_df = pd.read_csv(at.folder_path / "special_offer_product.csv")
    special_offer_df = pd.read_csv(at.folder_path / "special_offer.csv")
    store_df = pd.read_csv(at.folder_path / "store.csv")
    unit_measure_df = pd.read_csv(at.folder_path / "unit_measure.csv")
except Exception as e:
    print(e)
```

### ✓ A.1 Approach 1

```
# Columns available from each dataset

print("Person dataset columns:")
display(person_df.columns.tolist())

print("Customer dataset columns:")
display(customer_df.columns.tolist())

print("Sales order header dataset columns:")
display(sales_order_header_df.columns.tolist())

print("Sales order detail dataset columns:")
display(sales_order_detail_df.columns.tolist())
```

```

Person dataset columns:
['person_id',
 'name_style',
 'email_promotion',
 'person_type',
 'title',
 'first_name',
 'middle_name',
 'last_name',
 'suffix',
 'additional_contact_info']
Customer dataset columns:
['customer_id', 'person_id', 'store_id', 'territory_id', 'account_number']
Sales order header dataset columns:
['sales_order_id',
 'revision_number',
 'status',
 'online_order_flag',
 'customer_id',
 'sales_person_id',
 'territory_id',
 'currency_rate_id',
 'order_date',
 'due_date',
 'ship_date',
 'sales_order_number',
 'account_number',
 'sub_total',
 'tax_amount',
 'freight',
 'total_due']
Sales order detail dataset columns:
['sales_order_id',
 'sales_order_detail_id',
 'order_quantity',
 'product_id',
 'special_offer_id',
 'unit_price',
 'unit_price_discount',
 'line_total']

```

```

feature_selection_1_table = pd.DataFrame({
    "dataset": [
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_header",
        "sales_order_header",
        "sales_order_header",
        "sales_order_header",
        "customer",
        "customer",
        "person",
        "person"
    ],
    "column": [
        "line_total",
        "order_quantity",
        "unit_price",
        "unit_price_discount",
        "product_id",
        "sales_order_id",
        "customer_id",
        "order_date",
        "online_order_flag",
        "customer_id",
        "territory_id",
        "person_type",
        "email_promotion"
    ],
    "preparation_decision": [
        "Use to create order_total regression target",
        "Aggregate into order-level quantity features",
        "Aggregate into order-level price features",
        "Aggregate into order-level discount features",
        "Use to create product variety feature",
        "Use as grouping/joining key only",
        "Use as joining key only",
        "Use to create time-based features and time split",
        "Use as order channel feature",
        "Use as joining key only",
        "Use as territory feature",

```

```

        "Use as categorical person feature",
        "Use as categorical marketing preference feature"
    ]
})

```

```
feature_selection_1_table
```

|    | dataset            | column              | preparation_decision                             |
|----|--------------------|---------------------|--------------------------------------------------|
| 0  | sales_order_detail | line_total          | Use to create order_total regression target      |
| 1  | sales_order_detail | order_quantity      | Aggregate into order-level quantity features     |
| 2  | sales_order_detail | unit_price          | Aggregate into order-level price features        |
| 3  | sales_order_detail | unit_price_discount | Aggregate into order-level discount features     |
| 4  | sales_order_detail | product_id          | Use to create product variety feature            |
| 5  | sales_order_header | sales_order_id      | Use as grouping/joining key only                 |
| 6  | sales_order_header | customer_id         | Use as joining key only                          |
| 7  | sales_order_header | order_date          | Use to create time-based features and time split |
| 8  | sales_order_header | online_order_flag   | Use as order channel feature                     |
| 9  | customer           | customer_id         | Use as joining key only                          |
| 10 | customer           | territory_id        | Use as territory feature                         |
| 11 | person             | person_type         | Use as categorical person feature                |
| 12 | person             | email_promotion     | Use as categorical marketing preference feature  |

```
feature_selection_1_insights = """
```

The selected regression business use case is to predict the total sales amount of an order. The target variable is `order_total`. The EDA notebooks showed that `sales_order_detail` is the main transaction dataset, while `sales_order_header`, `customer`, and `person` provide additional order, customer, territory, and person-level context. This preparation notebook does not repeat the full EDA analysis. Instead, it uses EDA findings to select useful features.

```
# DO NOT MODIFY THE CODE IN THIS CELL
```

```
print_tile(size="h3", key='feature_selection_1_insights', value=feature_selection_1_insights)
```

```
feature_selection_1_insights
```

**The selected regression business use case is to predict the total sales amount of an order. The target variable is `order_total`, which is created by aggregating `line_total` from `sales_order_detail` at the `sales_order_id` level. The EDA notebooks showed that `sales_order_detail` is the main transaction dataset, while `sales_order_header`, `customer`, and `person` provide additional order, customer, territory, and person-level context. These datasets are joined using the relationship path: `sales_order_detail` to `sales_order_header` using `sales_order_id`, `sales_order_header` to `customer` using `customer_id`, and `customer` to `person` using `person_id`. This preparation notebook does not repeat the full EDA analysis.**

## ✓ A.2 Approach 2

```

join_key_validation = pd.DataFrame({
    "dataset": [
        "person",
        "customer",
        "sales_order_header",
        "sales_order_detail"
    ],
    "main_key": [
        "person_id",
        "customer_id",
        "sales_order_id",
        "sales_order_id"
    ],
    "rows": [
        len(person_df),
        len(customer_df),
        len(sales_order_header_df),
        len(sales_order_detail_df)
    ],
    "unique_key_values": [
        person_df["person_id"].nunique(),
        customer_df["customer_id"].nunique(),
        sales_order_header_df["sales_order_id"].nunique(),
        sales_order_detail_df["sales_order_id"].nunique()
    ],
    "missing_key_values": [
        person_df["person_id"].isna().sum(),

```

```

customer_df["customer_id"].isna().sum(),
sales_order_header_df["sales_order_id"].isna().sum(),
sales_order_detail_df["sales_order_id"].isna().sum()
],
"duplicate_key_rows": [
    len(person_df) - person_df["person_id"].nunique(),
    len(customer_df) - customer_df["customer_id"].nunique(),
    len(sales_order_header_df) - sales_order_header_df["sales_order_id"].nunique(),
    len(sales_order_detail_df) - sales_order_detail_df["sales_order_id"].nunique()
],
"fully_duplicated_rows": [
    person_df.duplicated().sum(),
    customer_df.duplicated().sum(),
    sales_order_header_df.duplicated().sum(),
    sales_order_detail_df.duplicated().sum()
]
})

join_key_validation

```

|   | dataset            | main_key       | rows  | unique_key_values | missing_key_values | duplicate_key_rows | fully_duplicated |
|---|--------------------|----------------|-------|-------------------|--------------------|--------------------|------------------|
| 0 | person             | person_id      | 25163 | 19972             | 0                  | 5191               |                  |
| 1 | customer           | customer_id    | 14275 | 10331             | 0                  | 3944               |                  |
| 2 | sales_order_header | sales_order_id | 31465 | 31465             | 0                  | 0                  |                  |
| 3 | sales_order_detail | sales_order_id | 42100 | 15357             | 0                  | 26743              |                  |

```

relationship_validation = pd.DataFrame({
    "relationship": [
        "customer to person",
        "sales_order_header to customer",
        "sales_order_detail to sales_order_header"
    ],
    "left_dataset": [
        "customer",
        "sales_order_header",
        "sales_order_detail"
    ],
    "right_dataset": [
        "person",
        "customer",
        "sales_order_header"
    ],
    "join_key": [
        "person_id",
        "customer_id",
        "sales_order_id"
    ],
    "left_unique_values": [
        customer_df["person_id"].nunique(),
        sales_order_header_df["customer_id"].nunique(),
        sales_order_detail_df["sales_order_id"].nunique()
    ],
    "right_unique_values": [
        person_df["person_id"].nunique(),
        customer_df["customer_id"].nunique(),
        sales_order_header_df["sales_order_id"].nunique()
    ],
    "left_missing_key_values": [
        customer_df["person_id"].isna().sum(),
        sales_order_header_df["customer_id"].isna().sum(),
        sales_order_detail_df["sales_order_id"].isna().sum()
    ],
    "right_missing_key_values": [
        person_df["person_id"].isna().sum(),
        customer_df["customer_id"].isna().sum(),
        sales_order_header_df["sales_order_id"].isna().sum()
    ]
})

relationship_validation

```

|   | relationship                             | left_dataset       | right_dataset      | join_key       | left_unique_values | right_unique_values | left_mi |
|---|------------------------------------------|--------------------|--------------------|----------------|--------------------|---------------------|---------|
| 0 | customer to person                       | customer           | person             | person_id      | 9232               | 19972               |         |
| 1 | sales_order_header to customer           | sales_order_header | customer           | customer_id    | 19119              | 10331               |         |
| 2 | sales_order_detail to sales_order_header | sales_order_detail | sales_order_header | sales_order_id | 15357              | 31465               |         |

```
customer_territory_summary = (
    customer_df["territory_id"]
    .value_counts(dropna=False)
    .reset_index()
)

customer_territory_summary.columns = ["territory_id", "count"]
customer_territory_summary["percentage"] = (
    customer_territory_summary["count"] / len(customer_df) * 100
).round(2)

customer_territory_summary
```

|   | territory_id                         | count | percentage |
|---|--------------------------------------|-------|------------|
| 0 | 2ac923e9-3043-4f5e-bae0-ad28089bf187 | 5567  | 39.00      |
| 1 | e0e3bac4-790e-4617-84b3-be0c2bdb7070 | 3008  | 21.07      |
| 2 | 25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2 | 2851  | 19.97      |
| 3 | 0ad4c625-bb65-4376-8a41-0a65719b0db8 | 2849  | 19.96      |

```
feature_selection_2_insights = """
This section keeps only preparation-relevant validation checks and does not repeat the full EDA already complete
The main purpose is to confirm that the datasets can be safely joined using the required relationship path: cust
The customer EDA notebook did not strongly analyse territory_id, so a compact customer territory check is includ
Other EDA-style checks such as general missing value summaries, status distribution, online_order_flag distribut
"""
```

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_2_insights', value=feature_selection_2_insights)
```

feature\_selection\_2\_insights

**This section keeps only preparation-relevant validation checks and does not repeat the full EDA already completed in the customer and sales\_order\_header notebooks. The main purpose is to confirm that the datasets can be safely joined using the required relationship path: customer to person using person\_id, sales\_order\_header to customer using customer\_id, and sales\_order\_detail to sales\_order\_header using sales\_order\_id. The customer EDA notebook did not strongly analyse territory\_id, so a compact customer territory check is included here. territory\_id is useful because it can provide regional or customer-location context for predicting order sales amount. Other EDA-style checks such as general missing**

▼ A.n Approach "Leakage Identifier Review"

You can add more cells related to other approaches in this section

```
leakage_identifier_review = pd.DataFrame({
    "column": [
        "sales_order_id",
        "sales_order_detail_id",
        "customer_id",
        "person_id",
        "account_number",
        "store_id",
        "line_total",
        "sub_total",
        "tax_amount",
        "freight",
        "total_due",
        "revision_number",
        "sales_person_id",
        "currency_rate_id",
        "first_name",
        "middle_name",
        "last_name",
        "title",
        "suffix",
    ]
})
```

```

        "additional_contact_info"
    ],
    "decision": [
        "Use for grouping and joining only",
        "Use for counting line items only",
        "Use for joining only",
        "Use for joining only",
        "Drop because it is an identifier",
        "Drop because it has high missingness and limited value",
        "Use only to create the order_total target",
        "Drop because it directly relates to order value",
        "Drop because it directly relates to total_due",
        "Drop because it directly relates to total_due",
        "Drop because it directly reveals final order value",
        "Drop because the EDA showed very low variation",
        "Drop because of high missingness",
        "Drop because of high missingness",
        "Drop because name field has no modelling value",
        "Drop because name field has low modelling value",
        "Drop because name field has no modelling value",
        "Drop because mostly missing and low value",
        "Drop because mostly missing and low value",
        "Drop because mostly missing and low value"
    ]
}
})

```

leakage\_identifier\_review

|    | column                  | decision                                          |
|----|-------------------------|---------------------------------------------------|
| 0  | sales_order_id          | Use for grouping and joining only                 |
| 1  | sales_order_detail_id   | Use for counting line items only                  |
| 2  | customer_id             | Use for joining only                              |
| 3  | person_id               | Use for joining only                              |
| 4  | account_number          | Drop because it is an identifier                  |
| 5  | store_id                | Drop because it has high missingness and limit... |
| 6  | line_total              | Use only to create the order_total target         |
| 7  | sub_total               | Drop because it directly relates to order value   |
| 8  | tax_amount              | Drop because it directly relates to total_due     |
| 9  | freight                 | Drop because it directly relates to total_due     |
| 10 | total_due               | Drop because it directly reveals final order v... |
| 11 | revision_number         | Drop because the EDA showed very low variation    |
| 12 | sales_person_id         | Drop because of high missingness                  |
| 13 | currency_rate_id        | Drop because of high missingness                  |
| 14 | first_name              | Drop because name field has no modelling value    |
| 15 | middle_name             | Drop because name field has low modelling value   |
| 16 | last_name               | Drop because name field has no modelling value    |
| 17 | title                   | Drop because mostly missing and low value         |
| 18 | suffix                  | Drop because mostly missing and low value         |
| 19 | additional_contact_info | Drop because mostly missing and low value         |

```

feature_selection_n_insights = ""
provide an explanation on why you use this approach for feature selection and describe its results
"""

```

```

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_selection_n_insights', value=feature_selection_n_insights)

```

## A.z Final Selection of Features

```

target_column = "order_total"

selected_features_table = pd.DataFrame({
    "feature": [
        "number_of_line_items",
        "total_quantity",
    ]
})

```

```

        "average_quantity",
        "average_unit_price",
        "max_unit_price",
        "min_unit_price",
        "std_unit_price",
        "average_discount",
        "max_discount",
        "discounted_line_count",
        "unique_products",
        "unique_special_offers",
        "online_order_flag",
        "status",
        "territory_id",
        "order_year",
        "order_month",
        "order_dayofweek",
        "is_weekend",
        "person_type",
        "email_promotion"
    ],
    "source": [
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_detail",
        "sales_order_header",
        "sales_order_header",
        "sales_order_header/customer",
        "sales_order_header",
        "sales_order_header",
        "sales_order_header",
        "sales_order_header",
        "sales_order_header",
        "person",
        "person"
    ],
    "type": [
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "numeric",
        "categorical/binary",
        "categorical",
        "categorical",
        "time numeric",
        "time numeric",
        "time numeric",
        "time numeric",
        "binary",
        "categorical",
        "categorical"
    ]
}

selected_features_table

```



|    | feature               | source                      | type               |
|----|-----------------------|-----------------------------|--------------------|
| 0  | number_of_line_items  | sales_order_detail          | numeric            |
| 1  | total_quantity        | sales_order_detail          | numeric            |
| 2  | average_quantity      | sales_order_detail          | numeric            |
| 3  | average_unit_price    | sales_order_detail          | numeric            |
| 4  | max_unit_price        | sales_order_detail          | numeric            |
| 5  | min_unit_price        | sales_order_detail          | numeric            |
| 6  | std_unit_price        | sales_order_detail          | numeric            |
| 7  | average_discount      | sales_order_detail          | numeric            |
| 8  | max_discount          | sales_order_detail          | numeric            |
| 9  | discounted_line_count | sales_order_detail          | numeric            |
| 10 | unique_products       | sales_order_detail          | numeric            |
| 11 | unique_special_offers | sales_order_detail          | numeric            |
| 12 | online_order_flag     | sales_order_header          | categorical/binary |
| 13 | status                | sales_order_header          | categorical        |
| 14 | territory_id          | sales_order_header/customer | categorical        |
| 15 | order_year            | sales_order_header          | time numeric       |
| 16 | order_month           | sales_order_header          | time numeric       |
| 17 | order_dayofweek       | sales_order_header          | time numeric       |
| 18 | is_weekend            | sales_order_header          | binary             |
| 19 | person_type           | person                      | categorical        |
| 20 | email_promotion       | person                      | categorical        |

```
feature_selection_explanations = """
```

The final selected features are based on the EDA findings but the preparation notebook does not repeat the EDA. The selected input features describe order size, quantity, price level, discount behaviour, product variety, spe

The final prepared dataset is created at order level, with one row per sales order.

```
"""
```

```
# DO NOT MODIFY THE CODE IN THIS CELL
```

```
print_tile(size="h3", key='feature_selection_explanations', value=feature_selection_explanations)
```

```
feature_selection_explanations
```

**The final selected features are based on the EDA findings but the preparation notebook does not repeat the EDA. The target variable is order\_total, created by summing line\_total for each sales\_order\_id. The selected input features describe order size, quantity, price level, discount behaviour, product variety, special offer usage, order channel, order status, territory, time period, person type, and email promotion preference. The final prepared dataset is created at order level,**

## ✓ B. Data Cleaning

### ✓ B.1 Fixing "Fix Duplicate Records"

```
person_clean = person_df.copy()
customer_clean = customer_df.copy()
header_clean = sales_order_header_df.copy()
detail_clean = sales_order_detail_df.copy()

duplicate_check_before = pd.DataFrame({
    "dataset": [
        "person",
        "customer",
        "sales_order_header",
        "sales_order_detail"
    ],
    "rows_before": [
        len(person_clean),
        len(customer_clean),
        len(header_clean),
        len(detail_clean)
    ],
})
```

```

    "fully_duplicated_rows": [
        person_clean.duplicated().sum(),
        customer_clean.duplicated().sum(),
        header_clean.duplicated().sum(),
        detail_clean.duplicated().sum()
    ]
})

duplicate_check_before

```

|   | dataset            | rows_before | fully_duplicated_rows |
|---|--------------------|-------------|-----------------------|
| 0 | person             | 25163       | 5191                  |
| 1 | customer           | 14275       | 3944                  |
| 2 | sales_order_header | 31465       | 0                     |
| 3 | sales_order_detail | 42100       | 0                     |

```

person_clean = person_clean.drop_duplicates()
customer_clean = customer_clean.drop_duplicates()
header_clean = header_clean.drop_duplicates()
detail_clean = detail_clean.drop_duplicates()

duplicate_check_after = pd.DataFrame({
    "dataset": [
        "person",
        "customer",
        "sales_order_header",
        "sales_order_detail"
    ],
    "rows_after": [
        len(person_clean),
        len(customer_clean),
        len(header_clean),
        len(detail_clean)
    ],
    "fully_duplicated_rows_after": [
        person_clean.duplicated().sum(),
        customer_clean.duplicated().sum(),
        header_clean.duplicated().sum(),
        detail_clean.duplicated().sum()
    ]
})

duplicate_check_after

```

|   | dataset            | rows_after | fully_duplicated_rows_after |
|---|--------------------|------------|-----------------------------|
| 0 | person             | 19972      | 0                           |
| 1 | customer           | 10331      | 0                           |
| 2 | sales_order_header | 31465      | 0                           |
| 3 | sales_order_detail | 42100      | 0                           |

```

data_cleaning_1_explanations = """
Duplicate rows are removed from all datasets before joining. This step is part of data preparation, not repeated
"""

```

```

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_1_explanations', value=data_cleaning_1_explanations)

```

data\_cleaning\_1\_explanations

**Duplicate rows are removed from all datasets before joining. This step is part of data preparation, not repeated EDA. The purpose is to ensure that repeated records do not distort aggregation, joining, or model training.**

## B.2 Fixing "Prepare Person and Customer Datasets"

```

person_prepared_df = person_clean[
    [
        "person_id",
        "person_type",
        "email_promotion"
    ]
].copy()

```

```

person_prepared_df = person_prepared_df.drop_duplicates(subset=["person_id"])

person_prepared_df["person_type"] = person_prepared_df["person_type"].fillna("Unknown")
person_prepared_df["email_promotion"] = person_prepared_df["email_promotion"].fillna(
    person_prepared_df["email_promotion"].mode()[0]
)

person_prepared_df["person_type"] = person_prepared_df["person_type"].astype(str)
person_prepared_df["email_promotion"] = person_prepared_df["email_promotion"].astype(str)

person_prepared_df.head()

```

|   | person_id                            | person_type | email_promotion |
|---|--------------------------------------|-------------|-----------------|
| 0 | 0e7956c1-835b-487b-b4b3-b83d08811282 | IN          | 0.0             |
| 1 | 333a1b3e-04c5-4c0f-80db-d840f5cc6593 | SC          | 1.0             |
| 2 | ddfd0d6e-1be4-4c5a-9eaf-6bce0d98c302 | IN          | 2.0             |
| 3 | aef31e4a-052f-47dc-85bd-31b3312a09c9 | SC          | 0.0             |
| 4 | 63635d45-76e1-4f06-a25f-424bee9af615 | VC          | 0.0             |

```

customer_prepared_df = customer_clean[
    [
        "customer_id",
        "person_id",
        "territory_id"
    ]
].copy()

customer_prepared_df = customer_prepared_df.drop_duplicates(subset=["customer_id"])

customer_prepared_df["territory_id"] = customer_prepared_df["territory_id"].fillna("Unknown")
customer_prepared_df["territory_id"] = customer_prepared_df["territory_id"].astype(str)

customer_prepared_df.head()

```

|   | customer_id                          | person_id                            | territory_id                         |
|---|--------------------------------------|--------------------------------------|--------------------------------------|
| 0 | 4b5762b3-b7bf-4716-9bf9-5c1c9e1f375d | 092707ff-4088-4416-ac86-14491f3120c2 | 0ad4c625-bb65-4376-8a41-0a65719b0db8 |
| 1 | 26bfba89-84b7-4b5b-9bab-caf7a2791006 | fe043525-e0af-4578-a4b9-bfa7be28f4e4 | 25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2 |
| 2 | cf2cf4cc-fcce-4fe5-9400-78373420b525 | ee25bbaf-5cdd-4ed4-92fe-7136c4e803a9 | 0ad4c625-bb65-4376-8a41-0a65719b0db8 |
| 3 | 83f558b8-bdab-4781-b112-55d2c275ff52 | f0452f05-3690-4170-9fb0-fbf9995bbea7 | 2ac923e9-3043-4f5e-bae0-ad28089bf187 |
| 4 | 5edc53f4-7821-4a9b-9c27-a45dd2af713e | 16c2532d-adb6-4a33-b0ea-71b87d83a904 | 0ad4c625-bb65-4376-8a41-0a65719b0db8 |

```

person_customer_preparation_summary = pd.DataFrame({
    "dataset": ["person_prepared_df", "customer_prepared_df"],
    "rows": [len(person_prepared_df), len(customer_prepared_df)],
    "missing_values": [
        person_prepared_df.isna().sum().sum(),
        customer_prepared_df.isna().sum().sum()
    ],
    "duplicate_key_rows": [
        len(person_prepared_df) - person_prepared_df["person_id"].nunique(),
        len(customer_prepared_df) - customer_prepared_df["customer_id"].nunique()
    ]
})

person_customer_preparation_summary

```

|   | dataset              | rows  | missing_values | duplicate_key_rows |
|---|----------------------|-------|----------------|--------------------|
| 0 | person_prepared_df   | 19972 | 0              | 0                  |
| 1 | customer_prepared_df | 10331 | 174            | 0                  |

```

data_cleaning_2_explanations = """
The person dataset is reduced to person_id, person_type, and email_promotion based on the EDA findings. Name-rel
The customer dataset is reduced to customer_id, person_id, and territory_id. customer_id and person_id are used
"""

```

```

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_2_explanations', value=data_cleaning_2_explanations)

```

data\_cleaning\_2\_explanations

The person dataset is reduced to person\_id, person\_type, and email\_promotion based on the EDA findings. Name-related and contact fields are excluded because they do not add meaningful modelling value. The customer dataset is reduced to customer\_id, person\_id, and territory\_id. customer\_id and person\_id are used for joining, while territory\_id is kept as a

### ✓ B.3 Fixing "Prepare Sales Order Header"

```
header_clean["order_date"] = pd.to_datetime(
    header_clean["order_date"],
    errors="coerce"
)

header_prepared_df = header_clean[
    [
        "sales_order_id",
        "customer_id",
        "territory_id",
        "order_date",
        "online_order_flag",
        "status"
    ]
].copy()

header_prepared_df = header_prepared_df.drop_duplicates(subset=["sales_order_id"])

header_prepared_df["territory_id"] = header_prepared_df["territory_id"].fillna("Unknown")
header_prepared_df["status"] = header_prepared_df["status"].fillna("Unknown")
header_prepared_df["online_order_flag"] = header_prepared_df["online_order_flag"].fillna(
    header_prepared_df["online_order_flag"].mode()[0]
)

header_prepared_df["order_year"] = header_prepared_df["order_date"].dt.year
header_prepared_df["order_month"] = header_prepared_df["order_date"].dt.month
header_prepared_df["order_dayofweek"] = header_prepared_df["order_date"].dt.dayofweek
header_prepared_df["is_weekend"] = header_prepared_df["order_dayofweek"].isin([5, 6]).astype(int)

for col in ["order_year", "order_month", "order_dayofweek"]:
    header_prepared_df[col] = header_prepared_df[col].fillna(
        header_prepared_df[col].mode()[0]
    )

header_prepared_df["is_weekend"] = header_prepared_df["is_weekend"].fillna(0)

header_prepared_df["territory_id"] = header_prepared_df["territory_id"].astype(str)
header_prepared_df["status"] = header_prepared_df["status"].astype(str)
header_prepared_df["online_order_flag"] = header_prepared_df["online_order_flag"].astype(str)

header_prepared_df.head()
```

|   | sales_order_id                       | customer_id                          | territory_id                         | order_date          | online_order_flag | status | order_year | order_month | order |
|---|--------------------------------------|--------------------------------------|--------------------------------------|---------------------|-------------------|--------|------------|-------------|-------|
| 0 | 1b9285b8-4501-4c92-adf7-ee74b4fa3d62 | 038a7b5e-5d15-4a1a-8305-e18131e402cc | 1e179cbb-7db9-4a66-aa94-e6fb9b3d613e | 2011-05-30 22:00:00 | 0.0               | 5.0    | 2011       |             | 5     |
| 1 | b28eec58-f64d-4211-b604-bc9e801fb523 | 75c8fd92-2eb9-4223-b92c-f7c66751ed52 | 1e179cbb-7db9-4a66-aa94-e6fb9b3d613e | 2011-05-30 22:00:00 | 0.0               | 5.0    | 2011       |             | 5     |
| 2 | 664be852-4095-4495-887a-4fc24bb299e6 | e5107fe5-b23a-46a4-8e49-bd195ef3b713 | c8b914d0-8c4c-498f-af9f-6954c90f45db | 2011-05-30 22:00:00 | 0.0               | 5.0    | 2011       |             | 5     |

```
header_preparation_summary = pd.DataFrame({
    "metric": [
        "Rows in prepared header dataset",
        "Unique sales_order_id values",
        "Missing sales_order_id values",
        "Missing customer_id values",
        "Missing territory_id values",
        "Missing order_date values"
    ],
    "value": [
        len(header_prepared_df),
        header_prepared_df["sales_order_id"].nunique(),
        header_prepared_df["sales_order_id"].isna().sum(),
    ]
})
```

```

        header_prepared_df["customer_id"].isna().sum(),
        header_prepared_df["territory_id"].isna().sum(),
        header_prepared_df["order_date"].isna().sum()
    ]
})

header_preparation_summary

```

|   | metric                          | value |
|---|---------------------------------|-------|
| 0 | Rows in prepared header dataset | 31465 |
| 1 | Unique sales_order_id values    | 31465 |
| 2 | Missing sales_order_id values   | 0     |
| 3 | Missing customer_id values      | 0     |
| 4 | Missing territory_id values     | 0     |
| 5 | Missing order_date values       | 0     |

```

data_cleaning_3_explanations = """
The sales_order_header dataset is prepared by keeping only useful non-leakage columns. sales_order_id and custom
Financial columns such as sub_total, tax_amount, freight, and total_due are excluded to avoid target leakage. re
"""

```

```

# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_3_explanations', value=data_cleaning_3_explanations)

```

data\_cleaning\_3\_explanations

**The sales\_order\_header dataset is prepared by keeping only useful non-leakage columns. sales\_order\_id and customer\_id are retained for joining. territory\_id, online\_order\_flag, and status are retained as categorical/context features. order\_date is used to create time-based features. Financial columns such as sub\_total, tax\_amount, freight, and total\_due are excluded to avoid target leakage. revision\_number, sales\_person\_id, and currency\_rate\_id are also**

## ✓ B.n Fixing "Prepare Sales Order Detail"

You can add more cells related to other issues in this section

```

numeric_detail_columns = [
    "sales_order_detail_id",
    "order_quantity",
    "unit_price",
    "unit_price_discount",
    "line_total"
]

for col in numeric_detail_columns:
    detail_clean[col] = pd.to_numeric(detail_clean[col], errors="coerce")

numeric_conversion_check = pd.DataFrame({
    "column": numeric_detail_columns,
    "missing_after_conversion": [
        detail_clean[col].isna().sum() for col in numeric_detail_columns
    ]
})

numeric_conversion_check

```

|   | column                | missing_after_conversion |
|---|-----------------------|--------------------------|
| 0 | sales_order_detail_id | 0                        |
| 1 | order_quantity        | 0                        |
| 2 | unit_price            | 0                        |
| 3 | unit_price_discount   | 0                        |
| 4 | line_total            | 0                        |

```

rows_before_invalid_filter = len(detail_clean)

detail_clean = detail_clean[
    (detail_clean["order_quantity"] > 0) &
    (detail_clean["unit_price"] > 0) &
    (detail_clean["unit_price_discount"] >= 0) &

```

```

(detail_clean["unit_price_discount"] <= 1) &
(detail_clean["line_total"] > 0)
].copy()

rows_after_invalid_filter = len(detail_clean)

invalid_transaction_summary = pd.DataFrame({
    "metric": [
        "Rows before invalid transaction filtering",
        "Rows after invalid transaction filtering",
        "Rows removed"
    ],
    "value": [
        rows_before_invalid_filter,
        rows_after_invalid_filter,
        rows_before_invalid_filter - rows_after_invalid_filter
    ]
})

invalid_transaction_summary

```

|   | metric                                    | value |
|---|-------------------------------------------|-------|
| 0 | Rows before invalid transaction filtering | 42100 |
| 1 | Rows after invalid transaction filtering  | 42100 |
| 2 | Rows removed                              | 0     |

```

order_detail_agg = (
    detail_clean
    .groupby("sales_order_id")
    .agg(
        order_total=("line_total", "sum"),
        number_of_line_items=("sales_order_detail_id", "count"),
        total_quantity=("order_quantity", "sum"),
        average_quantity=("order_quantity", "mean"),
        average_unit_price=("unit_price", "mean"),
        max_unit_price=("unit_price", "max"),
        min_unit_price=("unit_price", "min"),
        std_unit_price=("unit_price", "std"),
        average_discount=("unit_price_discount", "mean"),
        max_discount=("unit_price_discount", "max"),
        discounted_line_count=("unit_price_discount", lambda x: (x > 0).sum()),
        unique_products=("product_id", "nunique"),
        unique_special_offers=("special_offer_id", "nunique")
    )
    .reset_index()
)

order_detail_agg["std_unit_price"] = order_detail_agg["std_unit_price"].fillna(0)

print("Order-level detail dataset shape:", order_detail_agg.shape)
order_detail_agg.head()

```

Order-level detail dataset shape: (15357, 14)

|   | sales_order_id                       | order_total | number_of_line_items | total_quantity | average_quantity | average_unit_price | max_un: |
|---|--------------------------------------|-------------|----------------------|----------------|------------------|--------------------|---------|
| 0 | 0007f106-7b04-41f1-b56a-9900fb98d6b2 | 2354.98     | 2                    | 2.0            | 1.0              | 1177.490000        |         |
| 1 | 0009b614-678a-47b8-9529-ae4ee4ce6d87 | 1754.98     | 2                    | 2.0            | 1.0              | 877.490000         |         |
| 2 | 000ac468-67fd-41f6-93b8-0635928d3f7f | 1179.47     | 3                    | 3.0            | 1.0              | 393.156667         |         |
| 3 | 000c5e2f-e485-41ae-8e24-34d03eb623eb | 2419.06     | 2                    | 2.0            | 1.0              | 1209.530000        |         |
| 4 | 0016d0b9-3146-4c6e-adcc-3aff4921aacb | 2393.06     | 2                    | 2.0            | 1.0              | 1196.530000        |         |

Joining Datasets

```

model_df = order_detail_agg.merge(
    header_prepared_df,
    on="sales_order_id",
    how="left",
    suffixes=("", "_header")
)

model_df = model_df.merge(
    customer_prepared_df,
    on="customer_id",
    how="left",
    suffixes=("", "_customer")
)

model_df = model_df.merge(
    person_prepared_df,
    on="person_id",
    how="left"
)

print("Joined modelling dataset shape:", model_df.shape)
model_df.head()

```

Joined modelling dataset shape: (15357, 27)

|   | sales_order_id                       | order_total | number_of_line_items | total_quantity | average_quantity | average_unit_price | max_un: |
|---|--------------------------------------|-------------|----------------------|----------------|------------------|--------------------|---------|
| 0 | 0007f106-7b04-41f1-b56a-9900fb98d6b2 | 2354.98     | 2                    | 2.0            | 1.0              | 1177.490000        |         |
| 1 | 0009b614-678a-47b8-9529-aefee4ce6d87 | 1754.98     | 2                    | 2.0            | 1.0              | 877.490000         |         |
| 2 | 000ac468-67fd-41f6-93b8-0635928d3f7f | 1179.47     | 3                    | 3.0            | 1.0              | 393.156667         |         |
| 3 | 000c5e2f-e485-41ae-8e24-34d03eb623eb | 2419.06     | 2                    | 2.0            | 1.0              | 1209.530000        |         |
| 4 | 0016d0b9-3146-4c6e-adcc-3aff4921aacb | 2393.06     | 2                    | 2.0            | 1.0              | 1196.530000        |         |

```

if "territory_id_customer" in model_df.columns:
    model_df["territory_id"] = model_df["territory_id"].replace("nan", np.nan)
    model_df["territory_id_customer"] = model_df["territory_id_customer"].replace("nan", np.nan)

    model_df["territory_id"] = model_df["territory_id"].fillna(
        model_df["territory_id_customer"]
    )

    model_df = model_df.drop(columns=["territory_id_customer"])

model_df["territory_id"] = model_df["territory_id"].fillna("Unknown").astype(str)
model_df["person_type"] = model_df["person_type"].fillna("Unknown").astype(str)
model_df["email_promotion"] = model_df["email_promotion"].fillna("Unknown").astype(str)
model_df["status"] = model_df["status"].fillna("Unknown").astype(str)
model_df["online_order_flag"] = model_df["online_order_flag"].fillna("Unknown").astype(str)

model_df.head()

```

|   | sales_order_id                       | order_total | number_of_line_items | total_quantity | average_quantity | average_unit_price | max_un: |
|---|--------------------------------------|-------------|----------------------|----------------|------------------|--------------------|---------|
| 0 | 0007f106-7b04-41f1-b56a-9900fb98d6b2 | 2354.98     | 2                    | 2.0            | 1.0              | 1177.490000        |         |
| 1 | 0009b614-678a-47b8-9529-ae4ee4ce6d87 | 1754.98     | 2                    | 2.0            | 1.0              | 877.490000         |         |
| 2 | 000ac468-67fd-41f6-93b8-0635928d3f7f | 1179.47     | 3                    | 3.0            | 1.0              | 393.156667         |         |
| 3 | 000c5e2f-e485-41ae-8e24-34d03eb623eb | 2419.06     | 2                    | 2.0            | 1.0              | 1209.530000        |         |
| 4 | 0016d0b9-3146-4c6e-adcc-3aff4921aacb | 2393.06     | 2                    | 2.0            | 1.0              | 1196.530000        |         |

```
join_quality_summary = pd.DataFrame({
    "metric": [
        "Rows in final joined dataset",
        "Unique sales_order_id values",
        "Missing customer_id values",
        "Missing person_id values",
        "Missing territory_id values",
        "Missing person_type values",
        "Missing email_promotion values",
        "Missing order_total values"
    ],
    "value": [
        len(model_df),
        model_df["sales_order_id"].nunique(),
        model_df["customer_id"].isna().sum(),
        model_df["person_id"].isna().sum(),
        model_df["territory_id"].isna().sum(),
        model_df["person_type"].isna().sum(),
        model_df["email_promotion"].isna().sum(),
        model_df["order_total"].isna().sum()
    ]
})
```

join\_quality\_summary

|   | metric                         | value |
|---|--------------------------------|-------|
| 0 | Rows in final joined dataset   | 15357 |
| 1 | Unique sales_order_id values   | 15357 |
| 2 | Missing customer_id values     | 0     |
| 3 | Missing person_id values       | 0     |
| 4 | Missing territory_id values    | 0     |
| 5 | Missing person_type values     | 0     |
| 6 | Missing email_promotion values | 0     |
| 7 | Missing order_total values     | 0     |

data\_cleaning\_n\_explanations = ""

The sales\_order\_detail dataset is aggregated from line-item level to order level. The target variable order\_total produces one row per sales order and creates transaction-based features describing order size, quantity, price. The prepared datasets are joined using the relationship path identified from the available datasets. sales\_order\_detail This creates a final order-level modelling dataset containing transaction, order, customer, territory, and person.

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_cleaning_n_explanations', value=data_cleaning_n_explanations)
```



data\_cleaning\_n\_explanations

The `sales_order_detail` dataset is aggregated from line-item level to order level. The target variable `order_total` is created by summing `line_total` for each `sales_order_id`. This produces one row per sales order and creates transaction-based features describing order size, quantity, price, discount behaviour, product variety, and special offer usage. The prepared datasets are joined using the relationship path identified from the available datasets. `sales_order_detail` is aggregated by `sales_order_id`, then joined to `sales_order_header` using `sales_order_id`. `sales_order_header` is joined to customer using

## ✓ C. Split Datasets

```
model_df = model_df.sort_values("order_date").reset_index(drop=True)

n_rows = len(model_df)
train_end = int(n_rows * 0.70)
val_end = int(n_rows * 0.85)

training_df = model_df.iloc[:train_end].copy()
validation_df = model_df.iloc[train_end:val_end].copy()
testing_df = model_df.iloc[val_end:].copy()

split_summary = pd.DataFrame({
    "dataset": ["training", "validation", "testing"],
    "rows": [len(training_df), len(validation_df), len(testing_df)],
    "percentage": [
        round(len(training_df) / n_rows * 100, 2),
        round(len(validation_df) / n_rows * 100, 2),
        round(len(testing_df) / n_rows * 100, 2)
    ],
    "start_date": [
        training_df["order_date"].min(),
        validation_df["order_date"].min(),
        testing_df["order_date"].min()
    ],
    "end_date": [
        training_df["order_date"].max(),
        validation_df["order_date"].max(),
        testing_df["order_date"].max()
    ],
    "mean_order_total": [
        training_df["order_total"].mean(),
        validation_df["order_total"].mean(),
        testing_df["order_total"].mean()
    ],
    "median_order_total": [
        training_df["order_total"].median(),
        validation_df["order_total"].median(),
        testing_df["order_total"].median()
    ]
})

split_summary
```

|   | dataset    | rows  | percentage | start_date          | end_date            | mean_order_total | median_order_total |
|---|------------|-------|------------|---------------------|---------------------|------------------|--------------------|
| 0 | training   | 10749 | 69.99      | 2011-05-30 22:00:00 | 2014-02-04 23:00:00 | 2253.428325      | 1174.480           |
| 1 | validation | 2304  | 15.00      | 2014-02-04 23:00:00 | 2014-04-10 22:00:00 | 1536.918902      | 588.960            |
| 2 | testing    | 2304  | 15.00      | 2014-04-10 22:00:00 | 2014-06-29 22:00:00 | 1184.834210      | 285.222            |

data\_splitting\_explanations = ""

The final dataset is split using a time-based split because `order_date` is available from `sales_order_header`. The data is split into 70% training, 15% validation, and 15% testing. The split is performed after aggregation a

# Do not modify this code

print\_tile(size="h3", key='data\_splitting\_explanations', value=data\_splitting\_explanations)

data\_splitting\_explanations

The final dataset is split using a time-based split because `order_date` is available from `sales_order_header`. This is more realistic for the business use case because the model is trained on earlier orders and evaluated on later orders. The data is split into 70% training, 15% validation, and 15% testing. The split is performed after aggregation and joining so that

## D. Feature Engineering

```
# DO NOT MODIFY THE CODE IN THIS CELL
# Create copy of datasets

try:
    training_df_eng = training_df.copy()
    validation_df_eng = validation_df.copy()
    testing_df_eng = testing_df.copy()
except Exception as e:
    print(e)
```

### D.1 New Feature "has\_discount"

```
for dataset in [training_df_eng, validation_df_eng, testing_df_eng]:
    dataset["has_discount"] = (dataset["discounted_line_count"] > 0).astype(int)

training_df_eng[["discounted_line_count", "has_discount"]].head()
```

|   | discounted_line_count | has_discount |
|---|-----------------------|--------------|
| 0 | 0                     | 0            |
| 1 | 0                     | 0            |
| 2 | 0                     | 0            |
| 3 | 0                     | 0            |
| 4 | 0                     | 0            |

```
feature_engineering_1_explanations = """
The has_discount feature identifies whether an order contains at least one discounted line item. This gives the
"""
```

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_1_explanations', value=feature_engineering_1_explanations)
```

feature\_engineering\_1\_explanations

**The has\_discount feature identifies whether an order contains at least one discounted line item. This gives the model a**

### D.2 New Feature "product\_diversity\_ratio"

```
for dataset in [training_df_eng, validation_df_eng, testing_df_eng]:
    dataset["product_diversity_ratio"] = (
        dataset["unique_products"] / dataset["number_of_line_items"]
    )

training_df_eng[[
    "unique_products",
    "number_of_line_items",
    "product_diversity_ratio"
]].head()
```

|   | unique_products | number_of_line_items | product_diversity_ratio |
|---|-----------------|----------------------|-------------------------|
| 0 | 1               | 1                    | 1.0                     |
| 1 | 1               | 1                    | 1.0                     |
| 2 | 1               | 1                    | 1.0                     |
| 3 | 1               | 1                    | 1.0                     |
| 4 | 1               | 1                    | 1.0                     |

```
feature_engineering_2_explanations = """
The product_diversity_ratio feature measures the variety of products in an order relative to the number of line
"""
```

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_2_explanations', value=feature_engineering_2_explanations)
```

feature\_engineering\_2\_explanations

**The product\_diversity\_ratio feature measures the variety of products in an order relative to the number of line items. This helps distinguish between orders containing many different products and orders that are less diverse.**

### ✓ D.3 New Feature "unit\_price\_range"

```
for dataset in [training_df_eng, validation_df_eng, testing_df_eng]:
    dataset["unit_price_range"] = dataset["max_unit_price"] - dataset["min_unit_price"]

training_df_eng[[
    "max_unit_price",
    "min_unit_price",
    "unit_price_range"
]].head()
```

|   | max_unit_price | min_unit_price | unit_price_range |
|---|----------------|----------------|------------------|
| 0 | 3399.99        | 3399.99        | 0.0              |
| 1 | 3399.99        | 3399.99        | 0.0              |
| 2 | 3578.27        | 3578.27        | 0.0              |
| 3 | 3399.99        | 3399.99        | 0.0              |
| 4 | 3374.99        | 3374.99        | 0.0              |

```
feature_engineering_3_explanations = """
The unit_price_range feature measures the difference between the highest and lowest unit price within each order
"""
```

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_3_explanations', value=feature_engineering_3_explanations)
```

feature\_engineering\_3\_explanations

**The unit\_price\_range feature measures the difference between the highest and lowest unit price within each order. This captures the price spread of products in the order.**

### ✓ D.n New Feature: "special\_offer\_diversity\_ratio"

You can add more cells related to new features in this section

```
for dataset in [training_df_eng, validation_df_eng, testing_df_eng]:
    dataset["special_offer_diversity_ratio"] = (
        dataset["unique_special_offers"] / dataset["number_of_line_items"]
    )

training_df_eng[[
    "unique_special_offers",
    "number_of_line_items",
    "special_offer_diversity_ratio"
]].head()
```

|   | unique_special_offers | number_of_line_items | special_offer_diversity_ratio |
|---|-----------------------|----------------------|-------------------------------|
| 0 | 1                     | 1                    | 1.0                           |
| 1 | 1                     | 1                    | 1.0                           |
| 2 | 1                     | 1                    | 1.0                           |
| 3 | 1                     | 1                    | 1.0                           |
| 4 | 1                     | 1                    | 1.0                           |

```
feature_engineering_n_explanations = """
The special_offer_diversity_ratio feature measures the number of unique special offers relative to the number of
"""
```

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='feature_engineering_n_explanations', value=feature_engineering_n_explanations)
```

feature\_engineering\_n\_explanations

**The special\_offer\_diversity\_ratio feature measures the number of unique special offers relative to the number of line items. This gives the model a relative measure of offer variety rather than a simple count.**

## ✓ E. Data Preparation for Modeling

```
# DO NOT MODIFY THE CODE IN THIS CELL
# Create copy of datasets

try:
    X_train = training_df_eng.copy()
    X_val = validation_df_eng.copy()
    X_test = testing_df_eng.copy()
except Exception as e:
    print(e)
```

### ✓ E.1 Data Transformation

```
target_column = "order_total"

print("Training shape:", X_train.shape)
print("Validation shape:", X_val.shape)
print("Testing shape:", X_test.shape)
```

```
Training shape: (10749, 41)
Validation shape: (2304, 41)
Testing shape: (2304, 41)
```

```
# Create target copies from the engineered datasets
```

```
target_column = "order_total"

y_train = training_df_eng[[target_column]].copy()
y_val = validation_df_eng[[target_column]].copy()
y_test = testing_df_eng[[target_column]].copy()

print("y_train shape:", y_train.shape)
print("y_val shape:", y_val.shape)
print("y_test shape:", y_test.shape)
```

```
y_train shape: (10749, 1)
y_val shape: (2304, 1)
y_test shape: (2304, 1)
```

```
data_transformation_1_explanations = ""
```

The target variable is order\_total. The target column is kept inside the saved CSV files so that the model train

```
# DO NOT MODIFY THE CODE IN THIS CELL
```

```
print_tile(size="h3", key='data_transformation_1_explanations', value=data_transformation_1_explanations)
```

data\_transformation\_1\_explanations

**The target variable is order\_total. The target column is kept inside the saved CSV files so that the model training notebook can easily separate the target from the input features.**

### ✓ E.2 Data Transformation

```
log_transform_columns = [
    "order_total",
    "total_quantity",
    "average_unit_price",
    "max_unit_price",
    "min_unit_price",
    "std_unit_price",
    "unit_price_range"
]

for dataset in [X_train, X_val, X_test]:
    for col in log_transform_columns:
        if col in dataset.columns:
            dataset[f"log_{col}"] = np.log1p(dataset[col])
```

```
X_train[[col for col in X_train.columns if col.startswith("log_")]].head()
```

|   | log_order_total | log_total_quantity | log_average_unit_price | log_max_unit_price | log_min_unit_price | log_std_un |
|---|-----------------|--------------------|------------------------|--------------------|--------------------|------------|
| 0 | 8.131822        | 0.693147           | 8.131822               | 8.131822           | 8.131822           |            |
| 1 | 8.131822        | 0.693147           | 8.131822               | 8.131822           | 8.131822           |            |
| 2 | 8.182914        | 0.693147           | 8.182914               | 8.182914           | 8.182914           |            |
| 3 | 8.131822        | 0.693147           | 8.131822               | 8.131822           | 8.131822           |            |
| 4 | 8.124444        | 0.693147           | 8.124444               | 8.124444           | 8.124444           |            |

```
data_transformation_2_explanations = """
Log-transformed versions of skewed numerical variables are created because the EDA showed that sales and price-r
"""
```

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_transformation_2_explanations', value=data_transformation_2_explanations)
```

```
data_transformation_2_explanations
```

**Log-transformed versions of skewed numerical variables are created because the EDA showed that sales and price-related values are right-skewed. The original columns are kept, and log-transformed versions are added for modelling**

### ✓ E.3 Data Transformation

```
categorical_columns = [
    "online_order_flag",
    "status",
    "territory_id",
    "person_type",
    "email_promotion"
]

categorical_columns = [col for col in categorical_columns if col in X_train.columns]

X_train["dataset_split"] = "train"
X_val["dataset_split"] = "val"
X_test["dataset_split"] = "test"

combined_df = pd.concat(
    [X_train, X_val, X_test],
    axis=0,
    ignore_index=True
)

combined_df = pd.get_dummies(
    combined_df,
    columns=categorical_columns,
    drop_first=False
)

X_train = combined_df[combined_df["dataset_split"] == "train"].drop(columns=["dataset_split"]).copy()
X_val = combined_df[combined_df["dataset_split"] == "val"].drop(columns=["dataset_split"]).copy()
X_test = combined_df[combined_df["dataset_split"] == "test"].drop(columns=["dataset_split"]).copy()

print("X_train shape after encoding:", X_train.shape)
print("X_val shape after encoding:", X_val.shape)
print("X_test shape after encoding:", X_test.shape)

X_train.head()
```

```
X_train shape after encoding: (10749, 41)
X_val shape after encoding: (2304, 41)
X_test shape after encoding: (2304, 41)
```

|   | order_total | number_of_line_items | total_quantity | average_quantity | average_unit_price | max_unit_price | min_un: |
|---|-------------|----------------------|----------------|------------------|--------------------|----------------|---------|
| 0 | 3399.99     | 1                    | 1.0            | 1.0              | 3399.99            | 3399.99        |         |
| 1 | 3399.99     | 1                    | 1.0            | 1.0              | 3399.99            | 3399.99        |         |
| 2 | 3578.27     | 1                    | 1.0            | 1.0              | 3578.27            | 3578.27        |         |
| 3 | 3399.99     | 1                    | 1.0            | 1.0              | 3399.99            | 3399.99        |         |
| 4 | 3374.99     | 1                    | 1.0            | 1.0              | 3374.99            | 3374.99        |         |

```
data_transformation_3_explanations = """
Categorical variables are one-hot encoded so that they can be used by regression models. The train, validation,
"""
```

```
# DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='data_transformation_3_explanations', value=data_transformation_3_explanations)
```

```
data_transformation_3_explanations
```

**Categorical variables are one-hot encoded so that they can be used by regression models. The train, validation, and test sets are temporarily combined before encoding so that all splits have the same final columns.**

## ✓ E.n Fixing "<describe\_issue\_here>"

You can add more cells related to data preparation in this section

```
# Remove target, identifiers, leakage columns, and raw date columns from feature datasets
```

```
columns_to_drop = [
    "order_total",
    "log_order_total",
    "sales_order_id",
    "sales_order_detail_id",
    "customer_id",
    "person_id",
    "order_date",
    "sub_total",
    "tax_amount",
    "freight",
    "total_due",
    "account_number",
    "store_id",
    "revision_number",
    "sales_person_id",
    "currency_rate_id",
    "due_date",
    "ship_date",
    "first_name",
    "middle_name",
    "last_name",
    "title",
    "suffix",
    "additional_contact_info"
]

for dataset in [X_train, X_val, X_test]:
    dataset.drop(columns=columns_to_drop, inplace=True, errors="ignore")

print("Final X_train shape:", X_train.shape)
print("Final X_val shape:", X_val.shape)
print("Final X_test shape:", X_test.shape)

print("Final y_train shape:", y_train.shape)
print("Final y_val shape:", y_val.shape)
print("Final y_test shape:", y_test.shape)

X_train.head()
```

```
Final X_train shape: (10749, 39)
Final X_val shape: (2304, 39)
Final X_test shape: (2304, 39)
Final y_train shape: (10749, 1)
Final y_val shape: (2304, 1)
Final y_test shape: (2304, 1)
```

|   | number_of_line_items | total_quantity | average_quantity | average_unit_price | max_unit_price | min_unit_price | std |
|---|----------------------|----------------|------------------|--------------------|----------------|----------------|-----|
| 0 | 1                    | 1.0            | 1.0              | 3399.99            | 3399.99        | 3399.99        |     |
| 1 | 1                    | 1.0            | 1.0              | 3399.99            | 3399.99        | 3399.99        |     |
| 2 | 1                    | 1.0            | 1.0              | 3578.27            | 3578.27        | 3578.27        |     |
| 3 | 1                    | 1.0            | 1.0              | 3399.99            | 3399.99        | 3399.99        |     |
| 4 | 1                    | 1.0            | 1.0              | 3374.99            | 3374.99        | 3374.99        |     |

```
final_validation_summary = pd.DataFrame({
    "dataset": ["X_train", "X_val", "X_test", "y_train", "y_val", "y_test"],
    "rows": [
        len(X_train),
        len(X_val),
        len(X_test),
        len(y_train),
        len(y_val),
        len(y_test)
    ],
    "columns": [
        X_train.shape[1],
        X_val.shape[1],
        X_test.shape[1],
        y_train.shape[1],
        y_val.shape[1],
        y_test.shape[1]
    ],
    "missing_values": [
        X_train.isna().sum().sum(),
        X_val.isna().sum().sum(),
        X_test.isna().sum().sum(),
        y_train.isna().sum().sum(),
        y_val.isna().sum().sum(),
        y_test.isna().sum().sum()
    ],
    "duplicated_rows": [
        X_train.duplicated().sum(),
        X_val.duplicated().sum(),
        X_test.duplicated().sum(),
        y_train.duplicated().sum(),
        y_val.duplicated().sum(),
        y_test.duplicated().sum()
    ]
})
```

```
final_validation_summary
```

|   | dataset | rows  | columns | missing_values | duplicated_rows |
|---|---------|-------|---------|----------------|-----------------|
| 0 | X_train | 10749 | 39      | 0              | 1297            |
| 1 | X_val   | 2304  | 39      | 0              | 92              |
| 2 | X_test  | 2304  | 39      | 0              | 104             |
| 3 | y_train | 10749 | 1       | 0              | 9280            |
| 4 | y_val   | 2304  | 1       | 0              | 1645            |
| 5 | y_test  | 2304  | 1       | 0              | 1641            |

```
target_distribution_check = pd.DataFrame({
    "dataset": ["y_train", "y_val", "y_test"],
    "order_total_mean": [
        y_train["order_total"].mean(),
        y_val["order_total"].mean(),
        y_test["order_total"].mean()
    ],
    "order_total_median": [
        y_train["order_total"].median(),
        y_val["order_total"].median(),

```

